

Rozwiązywanie Problemu Przepływowego za pomocą Algorytmu Tabu Search

Kurs : Algorytmy Optymalizacji

Autorzy: Hubert Grzegorzewski 272579, Igor Frysiak 272548

Data: 06.06.2026

1. Wstęp

Problem harmonogramowania przepływowego (ang. *Flow Shop Scheduling Problem*, FSSP) stanowi jedno z klasycznych zagadnień optymalizacji kombinatorycznej, szeroko spotykane w systemach produkcyjnych i logistycznych. Należy on do klasy problemów NP-trudnych, co oznacza, że znalezienie rozwiązania optymalnego dla dużych instancji w czasie wielomianowym jest niemożliwe. Z tego powodu w praktyce inżynierskiej do rozwiązywania problemu FSSP wykorzystuje się algorytmy przybliżone – heurystyki oraz metaheurystyki.

Celem niniejszego projektu była implementacja metaheurystyki **Tabu Search** (Przeszukiwanie z Tabu) oraz zbadanie jej skuteczności na popularnych w literaturze instancjach testowych Taillarda. Przeprowadzono eksperymenty badające wpływ wybranych parametrów algorytmu na jakość uzyskiwanych rozwiązań oraz zbadano skalowalność algorytmu dla problemów o różnej złożoności.

2. Definicja Problemu

W projekcie rozpatrywany jest permutacyjny problem przepływowy (Permutation Flow Shop Scheduling Problem - PFSP). Polega on na wyznaczeniu kolejności wykonywania n zadań na m maszynach.

Założenia modelu:

1. Każde zadanie musi przejść przez wszystkie maszyny w dokładnie tej samej kolejności (od maszyny 1 do m).
2. Każda maszyna może przetwarzać tylko jedno zadanie w danym momencie.
3. Operacji nie można przerywać (ang. *non-preemptive*).
4. Kolejność zadań na każdej z maszyn jest identyczna (wariant permutacyjny).

Daną wejściową jest macierz czasów operacji P o wymiarach $m \times n$, gdzie $p_{i,j}$ oznacza czas wykonywania zadania j na maszynie i .

Funkcja celu:

Celem optymalizacji jest minimalizacja całkowitego czasu zakończenia wszystkich zadań, zwanego **Makespanem** (C_{max}). Dla zadanej permutacji zadań π , czas zakończenia zadania π_j na maszynie i obliczany jest rekurencyjnie:

$$C_{i,\pi_j} = C_{i,\pi_{j-1}} + p_{i,\pi_j}$$

przy czym C_{1,π_j} oraz C_{i,π_1} .

Poszukiwana jest taka permutacja π , dla której $C_{max}(\pi)$ osiąga wartość minimalną.

Do oceny jakości algorytmu wykorzystano referencyjne instancje Taillarda oraz metrykę **PRD** (Percentage Relative Deviation), która określa błąd procentowy względem najlepszego znanego rozwiązania (Upper Bound - UB):

$$PRD = \frac{C_{max} - UB}{UB} \cdot 100\%$$

3. Zastosowany Algorytm

Do rozwiązania problemu zastosowano metaheurystykę Tabu Search, która została zaimplementowana w języku Python. Aby zapewnić wysoką jakość wyników, system składa się z dwóch etapów: konstrukcyjnego (heurystyka NEH) oraz optymalizacyjnego (Tabu Search).

3.1. Rozwiązanie początkowe (Heurystyka NEH)

Zamiast rozpoczynać przeszukiwanie od losowej permutacji, zastosowano heurystykę **NEH** (Nawaz-Enscore-Ham).

1. Zadania są sortowane malejąco według sumarycznego czasu ich przetwarzania na wszystkich maszynach.
2. Zaczynając od pierwszego zadania z listy, kolejne zadania są iteracyjnie wstawiane w taką pozycję dotychczasowego podciągu, która gwarantuje najmniejszy chwilowy wzrost C_{max} .

Dzięki NEH algorytm Tabu Search startuje z punktu o relatywnie niskim błędzie PRD, co znacząco przyspiesza zbieżność.

3.2. Metaheurystyka Tabu Search

Główna pętla algorytmu opiera się na przeszukiwaniu sąsiedztwa rozwiązania bieżącego z wykorzystaniem pamięci (listy Tabu). Zaimplementowano następujące mechanizmy:

- **Sąsiedztwo:** Przebadano dwa typy ruchów:
 - **Swap** : zamiana miejscami dwóch dowolnych zadań w permutacji.
 - **Insert** : wyjęcie zadania ze swojej pozycji i wstawienie go w inne miejsce. (Dla problemu FSSP ten ruch generuje zazwyczaj lepsze sąsiedztwo strukturalne).
- **Kryterium oceny:** W każdej iteracji przeglądane jest całe sąsiedztwo, a do kolejnego kroku wybierany jest sąsiad z najmniejszym C_{max} – nawet jeśli jest on gorszy od rozwiązania bieżącego (pozwala to na wyjście z minimów lokalnych).
- **Pamięć Tabu:** Wykonany ruch jest zapisywany na listę Tabu na zadaną liczbę iteracji (tzw. *Tabu Tenure*). W tym czasie ponowne wykonanie tego ruchu jest zabronione.
- **Kryterium Aspiracji:** Ruch znajdujący się na liście Tabu może zostać wykonany, jeśli prowadzi do rozwiązania absolutnie najlepszego (globalnie najlepszego ze wszystkich dotychczas znalezionych).

4. Badania i Eksperymenty

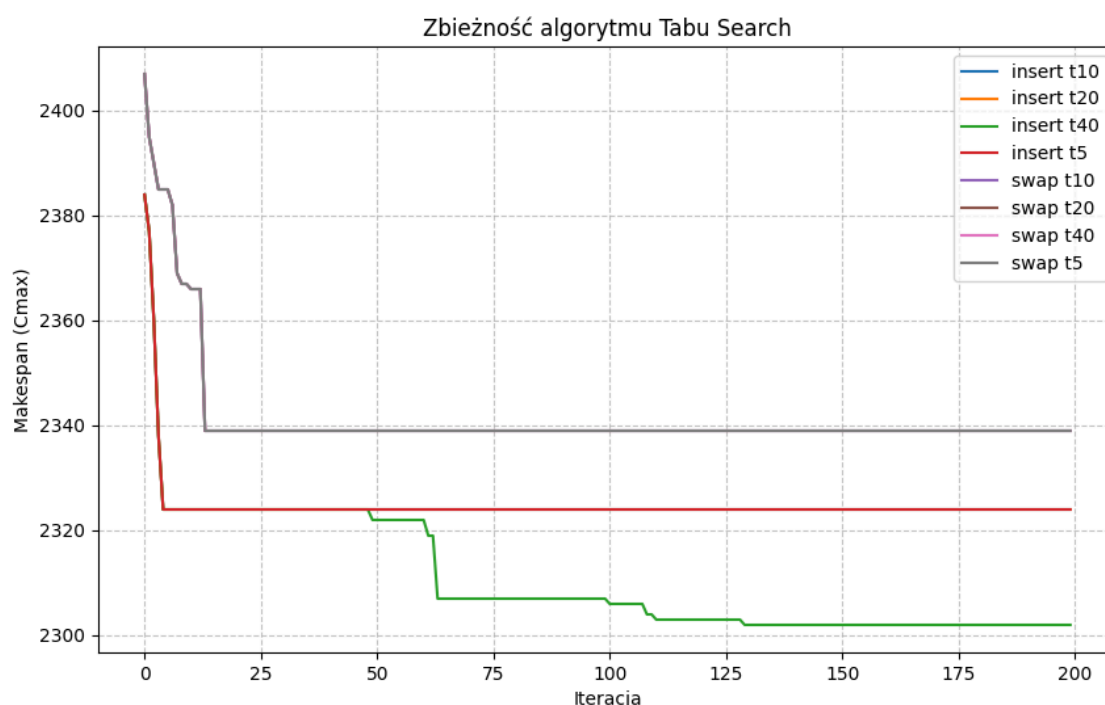
Badania przeprowadzono na instancjach z biblioteki Taillarda. Wyniki zebrano za pomocą dedykowanych skryptów ewaluacyjnych.

4.1. Badanie parametrów algorytmu (Wpływ sąsiedztwa i kadencji)

W pierwszej fazie badań skupiono się na doborze odpowiednich parametrów dla algorytmu. Przeprowadzono testy sprawdzając zachowanie algorytmu dla ruchu typu `Swap` oraz `Insert`, przy różnych długościach listy tabu.

Szczegółowe wyniki średniego błędu PRD w zależności od typu sąsiedztwa i rozmiaru listy zakazanej (Tabu Tenure) wykazały istotne różnice w zachowaniu algorytmu.

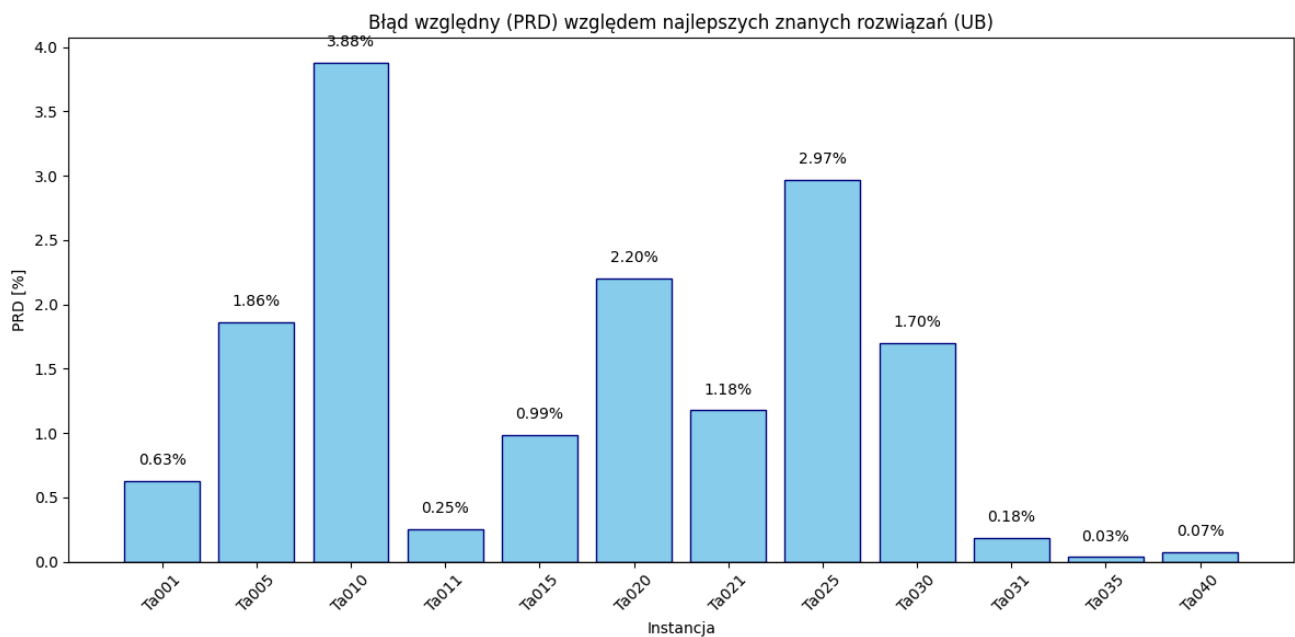
Na poniższym wykresie zbieżności widać wyraźnie, jak algorytm "wychodzi" z minimów lokalnych, akceptując gorsze rozwiązania, aby ostatecznie znaleźć lepsze optimum globalne.



(Rysunek 1. Zbieżność algorytmu Tabu Search dla różnych konfiguracji).

4.2. Wydajność globalna i skalowalność

Wykorzystując optymalne parametry, uruchomiono algorytm na przekrojowym zestawie instancji o różnych wymiarach (od 20x5 do 50x5).

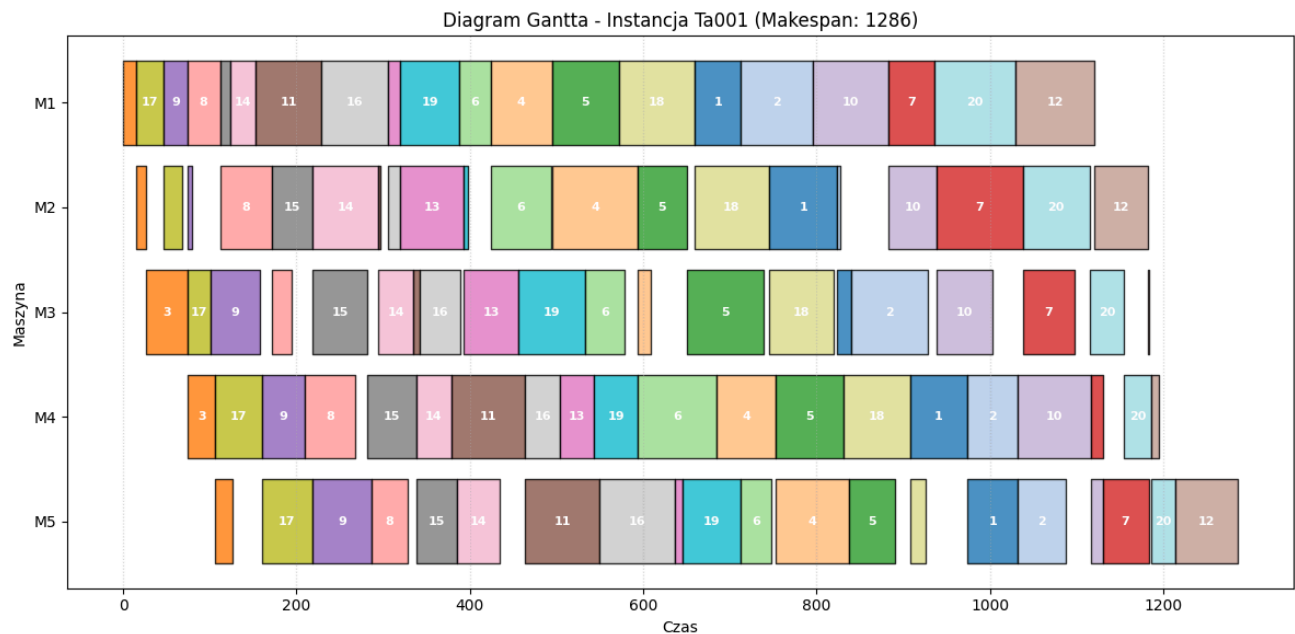


(Rysunek 2. Porównanie błędu PRD dla instancji o różnym rozmiarze).

Zaobserwowano, że algorytm radzi sobie bardzo dobrze dla instancji małych, jednak czas obliczeń i błąd odległości od optimum (UB) wzrastają dla instancji o większej liczbie maszyn i zadań.

4.3. Diagram Harmonogramu

Działanie algorytmu wizualizuje poniższy diagram Gantta wygenerowany dla przykładowej małej instancji (Ta001). Kolorami oznaczono poszczególne zadania procesowane na kolejnych maszynach.



(Rysunek 3. Diagram Gantta przedstawiający wyznaczony harmonogram).

5. Wnioski

Na podstawie przeprowadzonych badań, zebranych metryk oraz wykresów wyciągnięto następujące wnioski:

1. **Jakość sąsiedztwa:** Ruch typu `Insert` charakteryzuje się zazwyczaj znacznie wyższą skutecznością w eksploracji przestrzeni rozwiązań problemu FSSP niż ruch `Swap`. Wynika to ze specyfiki przesuwania "wąskich gardeł" na maszynach.
2. **Znaczenie NEH:** Wykorzystanie heurystyki NEH jako algorytmu startowego jest krytyczne dla osiągnięcia niskich wartości C_{max} . Algorytm Tabu Search startujący z losowej permutacji wymaga wielokrotnie więcej iteracji, aby osiągnąć podobną zbieżność.
3. **Pamięć algorytmu:** Dobór długości listy Tabu ma kluczowe znaczenie. Zbyt krótka lista powoduje zapętlanie się algorytmu wokół minimów lokalnych, podczas gdy zbyt długa nadmiernie ogranicza przestrzeń poszukiwań.
4. **Skalowalność:** Wraz ze wzrostem rozmiaru problemu (liczby zadań n i maszyn m) algorytm wymagał dłuższego czasu wykonania oraz miał trudność w dogonieniu optimum. Dla bardzo dużych instancji optymalizacja przeglądu sąsiedztwa (np. ograniczenie przeszukiwania całego sąsiedztwa na rzecz jego próbkowania) mogłaby stanowić interesujący kierunek dalszych prac rozwojowych.

Opracowana implementacja udowadnia, że Tabu Search jest wydajnym narzędziem do optymalizacji harmonogramów, pozwalającym na generowanie wyników bliskich najlepszym znanym rozwiązaniom literaturowym.